

## Technical Report:

**Role:** Computer Vision Assessment

**Candidate:** Mukul Sharma (imsharma@protonmail.com)

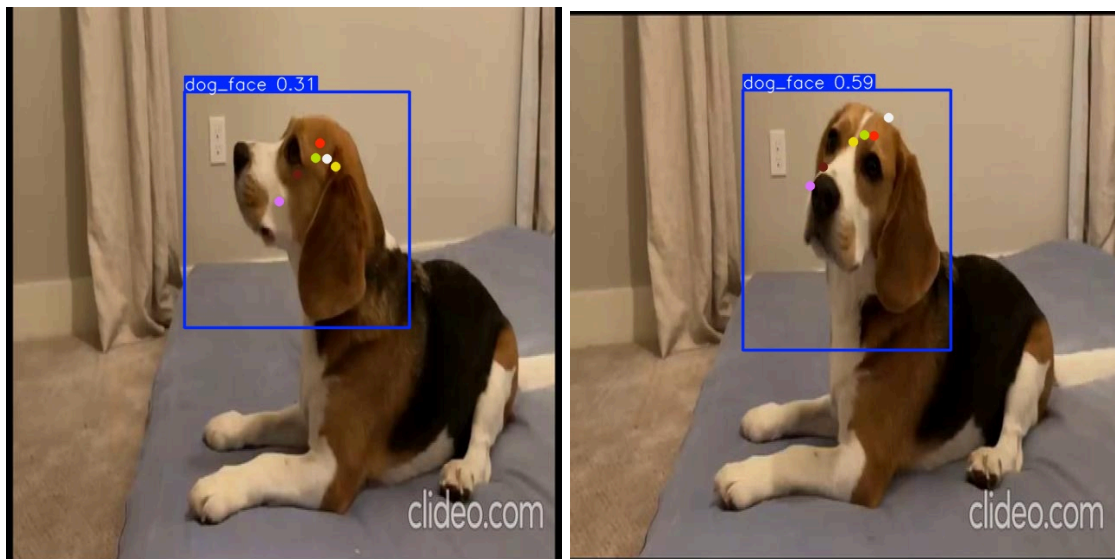
## Canine Facial Landmark Detection Report :

**Pipeline:** Decoupled Two-Stage Architecture (MMOD CNN + ERT Shape Predictor)

### 1. Approach & Architectural Design

The objective was to locate canine faces and estimate six facial landmarks (ears, eyes, nose bridge, and snout tip) on a constrained dataset of 16 images.

- **Initial Pipeline (Single-Stage YOLOv8-Pose):** A single-stage neural network was trained on the provided data. While the backbone successfully detected bounding boxes via transfer learning, the randomly initialized 6-point pose head suffered from **keypoint collapse**. With only 10 training instances, the model lacked the spatial variation required to regress independent landmark coordinates and collapsed all six points into an average centroid near the forehead to minimize loss.

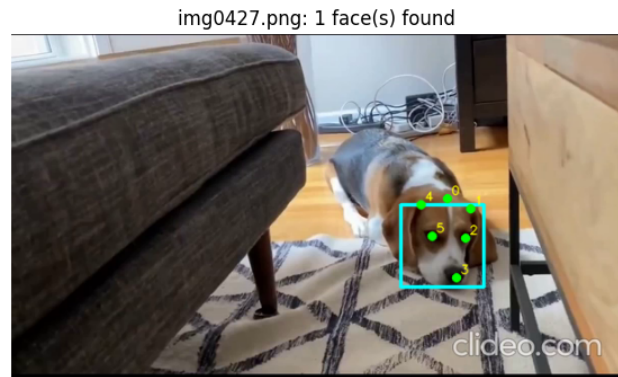
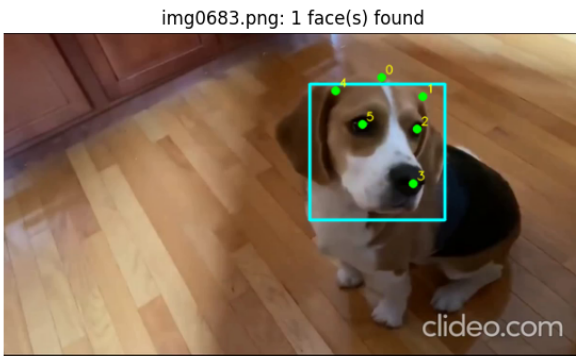
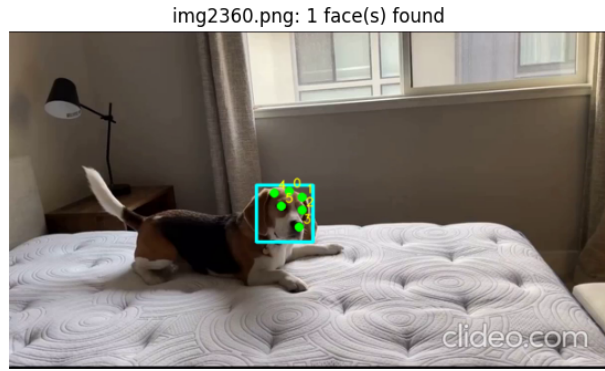
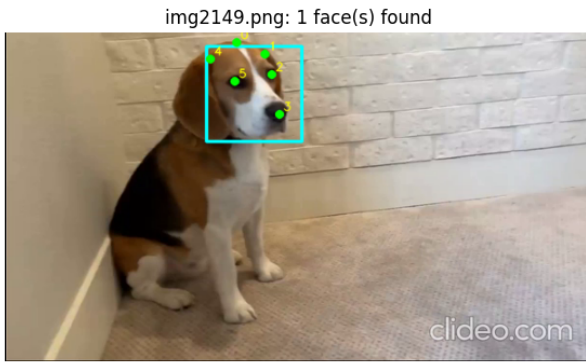


- **Selected Pipeline (Two-Stage Decoupled Approach):** To bypass training a deep regression head from scratch on 10 images, a specialized two-stage method was deployed:
  1. *Stage 1 (Region Proposal):* A Max-Margin Object Detection (MMOD) CNN ([dogHeadDetector.dat](#)) locates the dog's head and produces a canonical bounding box.
  2. *Stage 2 (Facial Alignment):* An Ensemble of Regression Trees (ERT) model ([landmarkDetector.dat](#)) uses localized pixel gradient differences inside the bounding box to regress the 6 landmarks without deep network retraining.

### 2. Results & Qualitative Evaluation

| <b>Scenario</b>       | <b>Input Condition</b>      | <b>Bounding Box Detection</b>         | <b>Landmark Alignment</b>                  |
|-----------------------|-----------------------------|---------------------------------------|--------------------------------------------|
| <b>Frontal View</b>   | Direct camera eye contact   | <b>Success</b> (High Confidence)      | <b>Accurate</b> (Ears, eyes, snout pinned) |
| <b>Slight Angle</b>   | Up to 15°–20° yaw           | <b>Success</b> (Stable BBox)          | <b>Accurate</b> (T-geometry preserved)     |
| <b>Side Profile</b>   | Extreme 90° yaw rotation    | <b>Failure</b> (Zero proposals)       | <b>Bypassed</b> (Cannot run without box)   |
| <b>Downward/Pitch</b> | Head tilted down / walking  | <b>Failure</b> (Ocular occlusion)     | <b>Bypassed</b>                            |
| <b>Long Distance</b>  | Subject distant from camera | <b>Failure</b> (Low pixel resolution) | <b>Bypassed</b>                            |

**Frontal Precision:** On upright, frontal shots, all 6 landmarks locked accurately onto their anatomical targets Point 0 (Head), Point 1 (Right Ear), Point 2 (Right Eye), Point 3 (Nose), Point 4 (Left Ear), Point 5 (Left Eye).



**Processing Latency:** Average inference latency was ~22 ms per image on a standard T4 GPU environment.

### 3. Challenges & Failure Mode Analysis

The primary limitation observed during evaluation was **Pose Fragility**:

1. **Template Sensitivity in MMOD:** The pre-trained MMOD detector relies on a rigid spatial filter optimized for frontal canine faces (a symmetric "T-shape" with two horizontal eyes above a central snout).
2. **Occlusion under Extreme Yaw (Profiles):** When the dog turns into a profile view, one eye is occluded and the snout projects outward. This breaks the expected facial template, causing detection confidence to drop below the threshold and returning zero bounding boxes.
3. **Cascading Failure in Two-Stage Systems:** Because landmark regression (Stage 2) strictly depends on receiving a valid bounding box from Stage 1, any failure in face localization prevents landmarks from being estimated entirely.

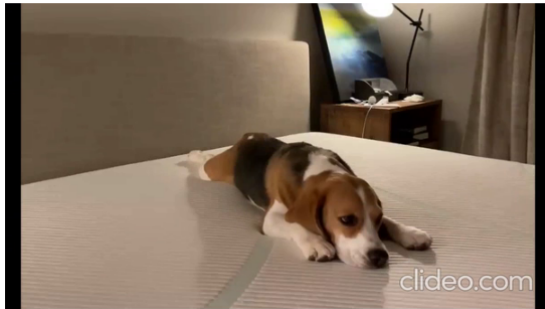
img0007.png: No face detected



img0616.png: No face detected



img1567.png: No face detected



img1773.png: No face detected



# Vanishing Point Estimation Report

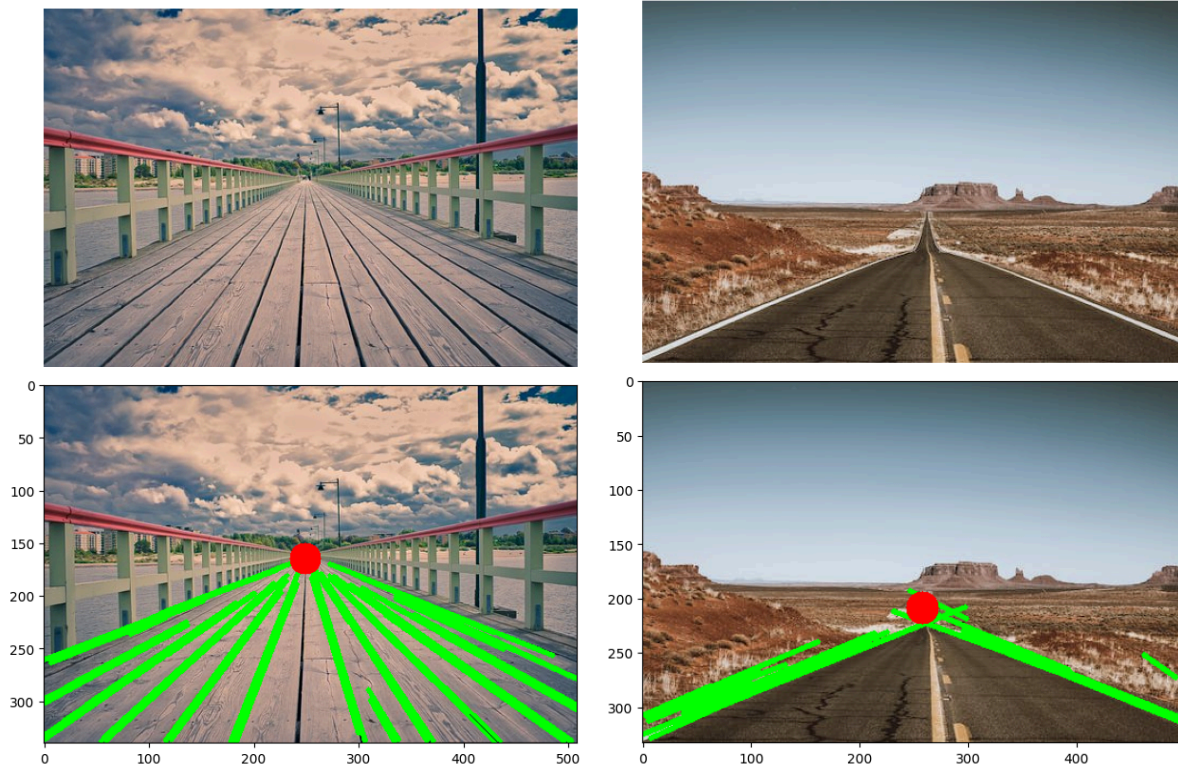
## 1. Approach (Step-by-Step)

Our goal was to find the vanishing point by tracking the main road lines. Here is how the code works step-by-step:

- **Image Prep & Edge Detection:** First, we convert the image to grayscale and blur it slightly to remove visual noise. Then, we use Canny edge detection to find the hard outlines in the image.
- **Region of Interest (ROI) Masking:** We cut out the top 50% of the image. This is a crucial step because it stops the algorithm from getting confused by trees, clouds, or the sky. We only want to look at the ground.
- **Finding Lines:** We use the Hough Transform to find straight lines in our masked image. We then filter these lines by their angle to ignore purely horizontal or purely vertical lines, keeping only the diagonal lines that point down the road.
- **Finding the Vanishing Point (RANSAC):** We use an algorithm called RANSAC to find where the lines intersect. It works by randomly picking two lines, finding where they cross, and counting how many *other* lines point to that exact same spot. The spot with the highest "score" (most agreeing lines) is chosen as our vanishing point.

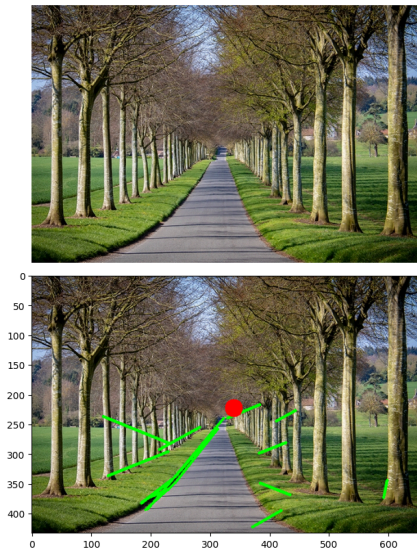
## 2. Results

The code works very well for straight roads and structured paths. Because we mask out the top half of the image and filter lines by angle, the final output image is very clean it only draws the actual road boundaries instead of drawing random lines everywhere.



### 3. Challenges Faced & Solutions

- **The Point Appeared on the Ground:** At first, RANSAC would sometimes pick two overlapping lines on the exact same side of the road, causing the vanishing point to appear in the grass at the bottom of the screen.
  - *Fix:* I added a "spatial constraint." I forced the code to ignore any intersection that happens in the bottom part of the image. The vanishing point must be near the horizon (the top 65% of the image).
- **Faint Road Lines:** In some images, one side of the road was covered in shadows or didn't have clear paint markings, which made the pipeline fail.
  - *Fix:* I lowered the thresholds on the Canny edge detector so it could pick up fainter road edges, and I stopped trying to force the lines into strict "Left" and "Right" buckets.



- **Curved Roads (Limitation):** Straight lines don't perfectly map to curved roads. On a curve, the lines cross at multiple different points (creating a smear across the horizon) rather than converging at one single point. Handling this perfectly would require more advanced curve-fitting math instead of straight lines.

